数值算法 Homework 02

姓名: 雍崔扬
学号: 21307140051
Due: 08:00 Mar. 3, 2025

Problem 1

Write a program to find

$$\inf_{x>0} \frac{x^3e^{2x}-1-3\ln(x)}{x}$$

What do you observe?

Solution:
定义函数 $f(\cdot)$:

$$\begin{aligned} f(x) &= \frac{x^3e^{2x}-1-3\ln(x)}{x} \\ &= x^2e^{2x} - \frac{1}{x} - \frac{3\ln(x)}{x} \end{aligned}$$

其一阶导数为:

$$\begin{aligned} f'(x) &= (2xe^{2x} + x^2\cdot 2e^{2x}) + \frac{1}{x^2} - 3\cdot\frac{\frac{1}{x}\cdot x - \ln(x)\cdot 1}{x^2} \\ &= (2x^2+2x)e^{2x} + \frac{3\ln(x)-2}{x^2} \end{aligned}$$

其二阶导数为:

$$\begin{aligned} f''(x) &= [(4x+2)e^{2x} + (2x^2+2x)\cdot 2e^{2x}] + \frac{\frac{3}{x}\cdot x^2 - (3\ln(x)-2)\cdot 2x}{x^4} \\ &= (4x^2+8x+2)e^{2x} + \frac{7-6\ln(x)}{x^3} \end{aligned}$$

- ① 当 $0<x<1$ 时，根据 $\ln(x)<0$ 可知 $f''(x)>0$
- ② $f''(1) = 14e^2+1>0$
- ③ 当 $x>1$ 时，根据 $\ln(x)<x-1$ 和 $e^x>x+1$ 可知:

$$\begin{aligned} f''(x) &= (4x^2+8x+2)e^{2x} + \frac{7-6\ln(x)}{x^3} \\ &> (4x^2+8x+2)(x+1) + \frac{7-6(x-1)}{x^3} \\ &= 4x^3+12x^2+10x+2+\frac{13}{x^3}-\frac{6}{x^2} \\ &> 4+12+10+2+0-6 \\ &= 22 \end{aligned}$$

综合 ①②③ 可知 $f''(x)>0$ 在 $(0,\infty)$ 上恒成立.
因此 $f'(x)$ 在 $(0,\infty)$ 上严格单调递增.

注意到:

$$
\begin{aligned}
f'(1) &= 4e^2 - 2 \\
&> 4 \cdot 4 - 2 \\
&> 0
\end{aligned}
$$

$$
\begin{aligned}
f'(0.5) &= \frac{3}{2}e - 8 - 12\ln(2) \\
&< \frac{9}{2} - 8 - 12\ln(2) \\
&< 0
\end{aligned}
$$

因此 $f'(x)$ 存在唯一的零点 $x_\star$，且 $x_\star$ 落在区间 $(0.5, 1)$ 中.
这说明 $f'(x)$ 在 $(0, x_\star)$ 上为负，在 $(x_\star, \infty)$ 上为正.
所以 $f(x)$ 在 $(0, x_\star)$ 上严格单调递减，在 $(x_\star, \infty)$ 上严格单调递增.
于是 $x_\star$ 是 $f(x)$ 在 $(0, \infty)$ 上的最小值点，即:

$$
\inf_{x>0} f(x) = \min_{x>0} f(x) = f(x_\star)
$$

因此问题归结为求解 $x_\star$ 的值，我们可以通过二分法近似计算它.
Matlab 代码如下:

```
% Define the function f(x) and g(x) = f'(x)
f = @(x) (x.^3 .* exp(2 .* x) - 1 - 3 .* log(x)) ./ x;
g = @(x) (2.*x.^2 + 2.*x) .* exp(2 .* x) + (3 .* log(x) - 2) ./ (x.^2);

% Set the initial interval (a, b)
a = 0.5;
b = 1;

% Set the stopping criterion (length of the interval)
tol_x = 1e-10;  % The tolerance for the root location
tol_y = 1e-10;

% Check if the function has opposite signs at the endpoints
g_a = g(a);
if g_a * g(b) > 0
    error('The function must have opposite signs at the endpoints a and b');
end

% Bisection method loop
while (b - a) > tol_x
    % Compute the midpoint
    c = (a + b) / 2;
    g_c = g(c);

    % Evaluate g(c)
    if abs(g_c) < tol_y
        break;  % If c is exactly the root, stop
    elseif g_a * g_c < 0
        b = c;  % If the root is in the left half
    else
        a = c;  % If the root is in the right half
        g_a = g_c;
    end
end

% The approximate root is at the midpoint
root = (a + b) / 2;
fprintf("The root of the derivative g(.) is approximately: %.10f\n", root);
```

```
fprintf("The minimum of the function f(.) is approximately: %.10f\n", f(root));
```

运行结果: (保留小数点后 10 位有效数字)

```
The root of the derivative g(.) is approximately: 0.6488441333
The minimum of the function f(.) is approximately: 2.0000000000
```

换言之:

$$x_\star \approx 0.6488441333$$
$$\min_{x>0} f(x) = f(x_\star) \approx 2.0000000000$$

What do you observe?
这道题告诉我们优化问题可以转化为导函数的求根问题来解决.

Problem 2

Use bisection and regula falsi, respectively, over the interval $[0, 1]$ to find the root of $x^{64} - 0.1 = 0$ with absolute accuracy 10^{-12}.
Visualize the convergence history of these methods in one figure.

Solution:
二分法取的是区间中点:

$$x_k := \frac{a_k + b_k}{2}$$

而试位法 (regula falsi) 取的是割线与水平坐标轴的交点:

$$x_k := b_k - \frac{f(b_k)(b_k - a_k)}{f(b_k) - f(a_k)} = \frac{f(b_k)a_k - f(a_k)b_k}{f(b_k) - f(a_k)}$$

二分法的 Matlab 代码:

```
% Function to implement Bisection Method
function [x, y] = bisection(a, b, tol_x, tol_y, max_iter, f)
    % Initialize arrays to store the history of x (roots) and y (function values)
    x = zeros(max_iter, 1);
    y = zeros(max_iter, 1);

    % Calculate initial function value at 'a'
    f_a = f(a);

    % Ensure the function has opposite signs at the endpoints
    if f_a * f(b) > 0
        error('The function must have opposite signs at the endpoints a and b');
    end

    % Perform the bisection method iteration
    for i = 1:max_iter
        % Compute the midpoint and evaluate the function at that point
        x(i) = (a + b) / 2;
        y(i) = f(x(i));

        % If the function value is close enough to zero or the interval is small enough, stop
        if abs(y(i)) < tol_y || (b - a) < tol_x
            break;
```

```
        elseif f_a * y(i) < 0
            % If the root is in the left half, update the right endpoint
            b = x(i);
        else
            % If the root is in the right half, update the left endpoint
            a = x(i);
            f_a = y(i);  % Update function value at the new left endpoint
        end
    end

    % Trim the unused elements from the arrays and return the history
    x = x(1:i);
    y = y(1:i);
end
```

试位法的 Matlab 代码:

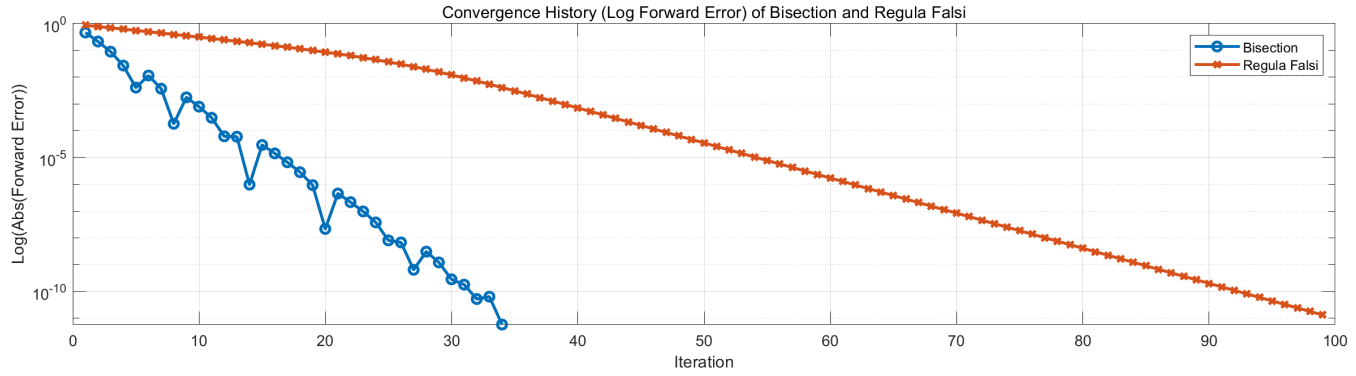
```
% Function to implement Regula Falsi Method
function [x, y] = regula_falsi(a, b, tol_x, tol_y, max_iter, f)
    % Initialize arrays to store the history of x (roots) and y (function values)
    x = zeros(max_iter, 1);
    y = zeros(max_iter, 1);

    % Calculate initial function values at 'a' and 'b'
    f_a = f(a);
    f_b = f(b);

    % Ensure the function has opposite signs at the endpoints
    if f_a * f_b > 0
        error('The function must have opposite signs at the endpoints a and b');
    end

    % Perform the regula falsi method iteration
    for i = 1:max_iter
        % Compute the new point using the Regula Falsi formula
        x(i) = (f_b * a - f_a * b) / (f_b - f_a);
        y(i) = f(x(i));

        % If the function value is close enough to zero or the interval is small enough, stop
        if abs(y(i)) < tol_y || (b - a) < tol_x
            break;
        elseif f_a * y(i) < 0
            % If the root is in the left half, update the right endpoint
            b = x(i);
            f_b = y(i);  % Update function value at the new right endpoint
        else
            % If the root is in the right half, update the left endpoint
            a = x(i);
            f_a = y(i);  % Update function value at the new left endpoint
        end
    end

    % Trim the unused elements from the arrays and return the history
    x = x(1:i);
    y = y(1:i);
end
```

函数调用:

```
% Define the function f(x)
f = @(x) x.^64 - 0.1;  % The function for which we are finding the root

% Set the initial interval [a, b] where we will search for the root
a = 0;  % Left endpoint of the interval
b = 1;  % Right endpoint of the interval

% Call the bisection method
[x_bisection, y_bisection] = bisection(a, b, 1e-10, 1e-10, 200, f);

% Call the regula falsi method
[x_falsi, y_falsi] = regula_falsi(a, b, 1e-10, 1e-10, 200, f);

% Approximate root
approx_root = x_bisection(end);
fprintf("Approximate root = %.10f\n", approx_root);

% Now we plot the log of the convergence history for both methods
% Compute the true root for reference, since we know it's around 0.1
true_root = 0.1^(1/64);

% Plot the convergence history for both methods:
% First Subplot: log(abs(x(i) - true_root)) for both methods
figure;
subplot(2,1,1);

% Plot Bisection method error in log scale
semilogy(1:length(x_bisection), abs(x_bisection - true_root), '-o', 'LineWidth', 2);
hold on;
% Plot Regula Falsi method error in log scale
semilogy(1:length(x_falsi), abs(x_falsi - true_root), '-x', 'LineWidth', 2);
xlabel('Iteration');
ylabel('Log(Abs(Forward Error))');
title('Convergence History (Log Forward Error) of Bisection and Regula Falsi');
legend('Bisection', 'Regula Falsi');
grid on;

% Second Subplot: log(abs(y(i))) for both methods
subplot(2,1,2);

% Plot Bisection method |y(i)| in log scale
semilogy(1:length(y_bisection), abs(y_bisection), '-o', 'LineWidth', 2);
hold on;
% Plot Regula Falsi method |y(i)| in log scale
semilogy(1:length(y_falsi), abs(y_falsi), '-x', 'LineWidth', 2);
xlabel('Iteration');
ylabel('Log(Abs(Backward Error))');
title('Convergence History (Log Backward Error) of Bisection and Regula Falsi');
legend('Bisection', 'Regula Falsi');
grid on;
```

运行结果:

```
Approximate root = 0.9646616199
```



Problem 3

When applying Newton's method to solve the equation $f(x) = 0$,
we usually require that $f'(x_\star) \neq 0$, i.e., the root $x_\star$ is a simple one.
Without such a condition, Newton's method is still applicable to find $x_\star$ while the convergence is no longer quadratic.
In the following let us assume that $f(x)$ is sufficiently smooth to avoid complications in theoretical analysis.

- (a) Use Newton's method to solve $1 + \cos(x) = 0$ around $x_0 = 3$ and plot the convergence history.
- (b) Let $x_\star$ be a root of $f(x)$ with multiplicity higher than one, i.e., $f(x_\star) = f'(x_\star) = 0$.
  Show that Newton's method converges (locally) linearly around $x_\star$.
- (c) Let $x_\star$ be a root of $f(x)$ with multiplicity $m \geq 1$, i.e., $f(x_\star) = f'(x_\star) = \cdots = f^{(m-1)}(x_\star) = 0 \neq f^{(m)}(x_\star)$.
  We can modify Newton's method as $x_{k+1} = x_k - \frac{m f(x_k)}{f'(x_k)}$ to achieve local quadratic convergence.
  Try to explain why such a modification improves the convergence.

Section (a)

Use Newton's method to solve $1 + \cos(x) = 0$ around $x_0 = 3$ and plot the convergence history.

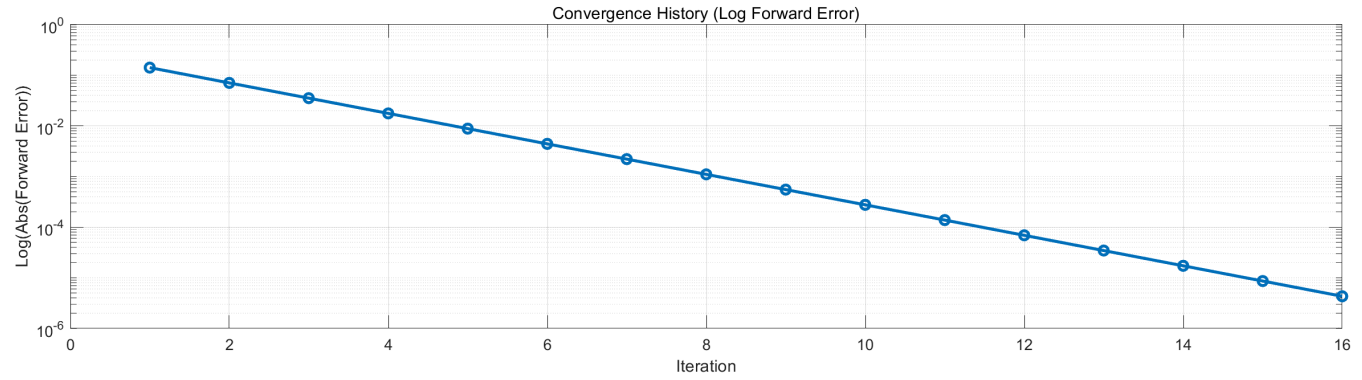
Solution:
定义 $f(x) = 1 + \cos(x)$，则 Newton 法的迭代格式为:

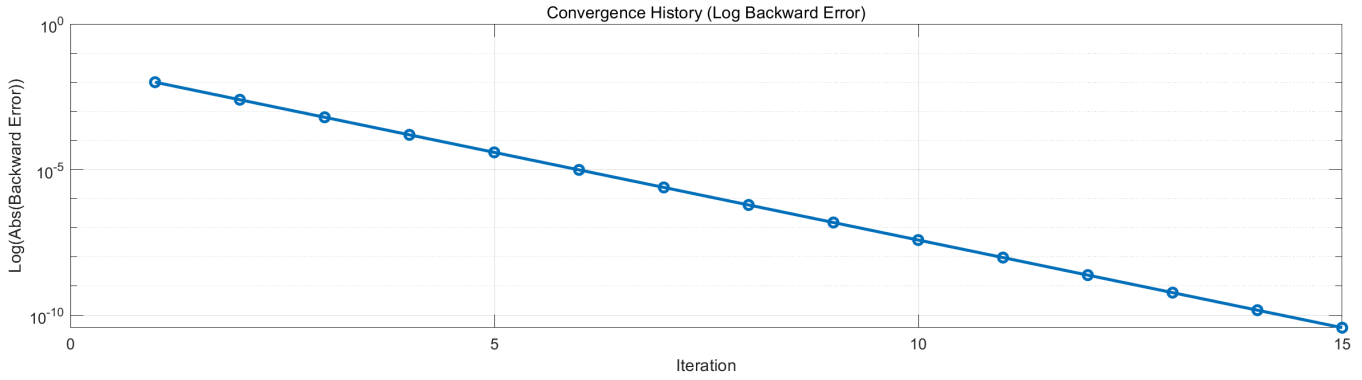
$$\begin{aligned} x_{k+1} &= x_k - \frac{f(x_k)}{f'(x_k)} \\ &= x_k - \frac{1 + \cos(x_k)}{-\sin(x_k)} \\ &= x_k + \frac{1 + \cos(x_k)}{\sin(x_k)} \end{aligned}$$

```
% Initialize the number of iterations (n) and arrays for x and y values
n = 20;  % Total number of iterations
```

```
x = zeros(n+1, 1);  % Array to store the approximations of x
y = zeros(n+1, 1);  % Array to store the function values y(i)

% Set the initial guess for x(1)
x(1) = 3;  % Initial guess for the solution

% Set the tolerance for convergence
tol = 1e-10;  % The stopping criterion (when y(i) becomes small enough)

% Iteration loop to calculate x and y using the given recurrence relation
for i = 1:n
    y(i) = 1 + cos(x(i));  % Calculate the function value at x(i)

    % Update x(i+1) using the recurrence relation: x(i+1) = x(i) + y(i) / sin(x(i))
    x(i+1) = x(i) + y(i) / sin(x(i));

    % Check if the backward error (y(i)) is smaller than the tolerance, and stop if true
    if abs(y(i)) < tol
        break;  % Break the loop if the backward error is small enough (convergence)
    end
end

% Trim the arrays to keep only the valid iterations
x = x(1:i+1);  % Keep only the valid approximations of x
y = y(1:i+1);  % Keep only the valid function values y

% Create a figure with two subplots for visualizing the convergence history
figure;

% First subplot: Log of the forward error |x - pi|
subplot(2, 1, 1);
semilogy(1:length(x), abs(x - pi), '-o', 'LineWidth', 2);  % Plot the forward error in log scale
xlabel('Iteration');  % Label for the x-axis (iterations)
ylabel('Log(Abs(Forward Error))');  % Label for the y-axis (log of the forward error)
title('Convergence History (Log Forward Error)');  % Title of the first subplot
grid on;  % Display grid for better readability

% Second subplot: Log of the backward error |y|
subplot(2, 1, 2);
semilogy(1:length(y), abs(y), '-o', 'LineWidth', 2);  % Plot the backward error in log scale
xlabel('Iteration');  % Label for the x-axis (iterations)
ylabel('Log(Abs(Backward Error))');  % Label for the y-axis (log of the backward error)
title('Convergence History (Log Backward Error)');  % Title of the second subplot
grid on;  % Display grid for better readability
```

运行结果:





这里的局部线性收敛性是因为 $x = \pi$ 是 $1 + \cos(x) = 0$ 的二重根:

$$
\begin{aligned}
1 + \cos(x) &= 1 + \cos(\pi + t) \quad (\text{denote } t = x - \pi) \\
&= 1 - \cos(t) \\
&= 1 - \left\{ \cos(0) - \sin(0)t - \frac{\cos(0)}{2!}t^2 + \frac{\sin(0)}{3!}t^3 + O(t^4) \right\} \\
&= 1 - 1 + 0 + \frac{1}{2}t^2 + O(t^4) \\
&= \frac{1}{2}t^2 + O(t^4)
\end{aligned}
$$

这表明 $t = 0$ 是 $1 + \cos(\pi + t)$ 的二重根，即 $x = \pi$ 是 $1 + \cos(x)$ 的二重根.

Section (b)

Let $x_\star$ be a root of $f(x)$ with multiplicity higher than one, i.e., $f(x_\star) = f'(x_\star) = 0$.
Show that Newton's method converges (locally) linearly around $x_\star$.

Solution: (reference1) & (reference2)
假设 $f(x)$ 充分光滑.
若根 $x_\star$ 的重数为 $m \geq 2$，则存在充分光滑的函数 $g(x)$ 满足:

$$
\begin{gathered}
f(x) = (x - x_\star)^m g(x) \quad (\forall\, x \in \mathrm{dom}(f)) \\
g(x_\star) \neq 0
\end{gathered}
$$

根据 Newton 法的迭代格式我们有:

$$
\begin{aligned}
x_{k+1}-x_\star &= x_k-\frac{f(x_k)}{f'(x_k)}-x_\star \\
&= x_k-\frac{(x_k-x_\star)^m g(x_k)}{m(x_k-x_\star)^{m-1}g(x_k)+(x_k-x_\star)^m g'(x_k)}-x_\star \\
&= \frac{mg(x_k)(x_k-x_\star)^m+g'(x_k)(x_k-x_\star)^{m+1}-g(x_k)(x_k-x_\star)^m}{mg(x_k)(x_k-x_\star)^{m-1}+g'(x_k)(x_k-x_\star)^m} \\
&= \frac{(m-1)g(x_k)(x_k-x_\star)^m+g'(x_k)(x_k-x_\star)^{m+1}}{mg(x_k)(x_k-x_\star)^{m-1}+g'(x_k)(x_k-x_\star)^m} \\
&= \frac{(m-1)g(x_k)+g'(x_k)(x_k-x_\star)}{mg(x_k)+g'(x_k)(x_k-x_\star)}(x_k-x_\star) \quad (\text{note that } g(x_k)=g(x_\star)+g'(x_\star)(x_k-x_\star)+O((x_k-x_\star)^2)) \\
&= \frac{(m-1)g(x_\star)+mg'(x_k)(x_k-x_\star)+O((x_k-x_\star)^2)}{mg(x_\star)+(m+1)g'(x_k)(x_k-x_\star)+O((x_k-x_\star)^2)}(x_k-x_\star) \\
&= \frac{m-1}{m}\cdot\frac{m(m-1)g(x_\star)+m^2g'(x_k)(x-x_\star)+O((x_k-x_\star)^2)}{m(m-1)g(x_\star)+(m-1)(m+1)g'(x_k)(x_k-x_\star)+O((x_k-x_\star)^2)} \\
&= \frac{m-1}{m}\left(1+\frac{g'(x_k)+O(x_k-x_\star)}{m(m-1)g(x_\star)+O(x_k-x_\star)}\right)(x_k-x_\star) \quad (\text{note that } g(x_\star)\neq 0) \\
&= \frac{m-1}{m}(x_k-x_\star)+O((x_k-x_\star)^2)
\end{aligned}
$$

这表明收敛速度是局部线性收敛.
(常数是 $\frac{m-1}{m}$，因此重数 m 越大，收敛速度越慢)

Section (c)

Let $x_\star$ be a root of $f(x)$ with multiplicity $m\geq 1$, i.e., $f(x_\star)=f'(x_\star)=\cdots=f^{(m-1)}(x_\star)=0\neq f^{(m)}(x_\star)$.
We can modify Newton's method as $x_{k+1}=x_k-\frac{mf(x_k)}{f'(x_k)}$ to achieve local quadratic convergence.
Try to explain why such a modification improves the convergence.

- **Lemma: (不动点迭代在特殊情况下的收敛性)**
  设 $g(\cdot)$ 二阶连续可导，考虑不动点迭代 $x_k=g(x_{k-1})$
  若 $g(x_\star)=x_\star$ 且 $g'(x_\star)\neq 0$，则不动点迭代局部二次收敛到 $x_\star$:

$$
\begin{aligned}
|x_k-x_\star| &= |g(x_{k-1})-g(x_\star)| \quad (\text{note that } x_k=g(x_{k-1}) \text{ and } x_\star=g(x_\star)) \\
&= \left|g(x_\star)+\frac{1}{2}g''(x_\star)(x_{k-1}-x_\star)^2+O((x_{k-1}-x_\star)^3)-g(x_\star)\right| \quad (\text{Taylor expansion, note that } g'(x_\star)=0) \\
&\leq \frac{1}{2}(|g''(x_\star)|+\varepsilon)|x_{k-1}-x_\star|^2 \text{ for some } \varepsilon \text{ so long as } x_{k-1} \text{ is close to } x_\star
\end{aligned}
$$

Solution: (reference)
假设 $f(x)$ 充分光滑.
若根 $x_\star$ 的重数为 $m\geq 2$，则存在充分光滑的函数 $g(x)$ 满足:

$$
\begin{gathered}
f(x)=(x-x_\star)^m g(x) \quad (\forall\, x\in \mathrm{dom}(f)) \\
g(x_\star)\neq 0
\end{gathered}
$$

根据修正后的 Newton 法的迭代格式我们有:

$$
\begin{aligned}
x_{k+1}-x_\star &= x_k-\frac{mf(x_k)}{f'(x_k)}-x_\star \\
&= x_k-\frac{m(x_k-x_\star)^m g(x_k)}{m(x_k-x_\star)^{m-1}g(x_k)+(x_k-x_\star)^m g'(x_k)}-x_\star \\
&= \frac{mg(x_k)(x_k-x_\star)^m+g'(x_k)(x_k-x_\star)^{m+1}-mg(x_k)(x_k-x_\star)^m}{mg(x_k)(x_k-x_\star)^{m-1}+g'(x_k)(x_k-x_\star)^m} \\
&= \frac{g'(x_k)(x_k-x_\star)^{m+1}}{mg(x_k)(x_k-x_\star)^{m-1}+g'(x_k)(x_k-x_\star)^m} \\
&= \frac{g'(x_k)(x_k-x_\star)}{mg(x_k)+g'(x_k)(x_k-x_\star)}(x_k-x_\star) \quad (\text{note that } g(x_k)=g(x_\star)+g'(x_\star)(x_k-x_\star)+O((x_k-x_\star)^2)) \\
&= \frac{g'(x_k)(x_k-x_\star)}{mg(x_\star)+(m+1)g'(x_k)(x_k-x_\star)+O((x_k-x_\star)^2)}(x_k-x_\star) \quad (\text{note that } g(x_\star)\neq 0) \\
&= O((x_k-x_\star)^2)
\end{aligned}
$$

这表明收敛速度是局部二次收敛.

另一种证明局部二次收敛的方法是将上述迭代格式纳入不动点迭代的框架下.
我们定义:

$$
\begin{aligned}
h(x) &:= x-\frac{mf(x)}{f'(x)} \quad (\text{note that } f(x)=(x-x_\star)^m g(x) \text{ where } g(x_\star)\neq 0) \\
&= x-\frac{m(x-x_\star)^m g(x)}{m(x-x_\star)^{m-1}g(x)+(x-x_\star)^m g'(x)} \\
&= \frac{m(x-x_\star)^m g(x)+(x-x_\star)^{m+1}g'(x)-m(x-x_\star)^m g(x)}{m(x-x_\star)^{m-1}g(x)+(x-x_\star)^m g'(x)}+x_\star \\
&= \frac{(x-x_\star)^{m+1}g'(x)}{m(x-x_\star)^{m-1}g(x)+(x-x_\star)^m g'(x)}+x_\star \\
&= \frac{(x-x_\star)^2 g'(x)}{mg(x)+(x-x_\star)g'(x)}+x_\star
\end{aligned}
$$

则修正后的 Newton 迭代格式等价于:

$$
x_{k+1}=x_k-\frac{mf(x_k)}{f'(x_k)}=h(x_k)
$$

根据 **Lemma** 可知:
只要证明 $\lim_{x\to x_\star}h(x)=x_\star$ 且 $\lim_{x\to x_\star}h'(x_\star)=0$，就可证明上述迭代格式的局部二次收敛性.

- 首先我们有:

$$
\lim_{x\to x_\star}h(x)=\lim_{x\to x_\star}\left\{\frac{(x-x_\star)^2g'(x)}{mg(x)+(x-x_\star)g'(x)}+x_\star\right\}=x_\star
$$

- 其次我们有:

$$
\begin{aligned}
h'(x) &= \frac{[2(x-x_\star)g'(x)+(x-x_\star)^2g''(x)][mg(x)+(x-x_\star)g'(x)]-(x-x_\star)^2g'(x)[(m+1)g'(x)+(x-x_\star)g''(x)]}{[mg(x)+(x-x_\star)g'(x)]^2} \\
&= \frac{2m(x-x_\star)g(x)g'(x)-(m-1)(x-x_\star)^2(g'(x))^2+m(x-x_\star)^2g(x)g''(x)}{[mg(x)+(x-x_\star)g'(x)]^2} \\
&\to 0 \quad (x\to x_\star)
\end{aligned}
$$

即 $\Rightarrow \lim_{x\to x_\star}h'(x)=0$

因此上述迭代格式具有局部二次收敛性.

Problem 4

In this exercise, you will determine the pH of rainwater by measuring the partial pressure of carbon dioxide (CO_2).
For simplicity, we suppose that the only chemical reactions in rainwater are:

$$\begin{aligned}\mathrm{H_2O} &\rightleftharpoons \mathrm{H^+} + \mathrm{OH^-}\\ \mathrm{CO_2} + \mathrm{H_2O} &\rightleftharpoons \mathrm{H^+} + \mathrm{HCO_3^-}\\ \mathrm{HCO_3^-} &\rightleftharpoons \mathrm{H^+} + \mathrm{CO_3^{2-}}\end{aligned}$$

The following nonlinear system of equations governs the chemistry of rainwater:

$$\begin{aligned}\kappa_{\text{water}} &= [\mathrm{H^+}][\mathrm{OH^-}]\\ \kappa_1 &= \frac{[\mathrm{H^+}][\mathrm{HCO_3^-}]}{\kappa_{\text{Henry}} \cdot p_{\mathrm{CO_2}}}\\ \kappa_2 &= \frac{[\mathrm{H^+}][\mathrm{CO_3^{2-}}]}{[\mathrm{HCO_3^-}]}\\ [\mathrm{H^+}] &= [\mathrm{OH^-}] + [\mathrm{HCO_3^-}] + 2[\mathrm{CO_3^{2-}}]\end{aligned}$$

where $\kappa_1 = 10^{-6.3}\mathrm{mol/L}, \kappa_2 = 10^{-10.3}\mathrm{mol/L}, \kappa_{\text{water}} = 10^{-14}(\mathrm{mol/L})^2$ are equilibrium constants (at $25^\circ\mathrm{C}$), and $\kappa_{\text{Henry}} = 10^{-1.46}\mathrm{mol}/(\mathrm{L}\cdot\mathrm{atm})$ is Henry's constant for $\mathrm{CO_2}$ dissolving in water (at $25^\circ\mathrm{C}$)

Let us suppose $p_{\mathrm{CO_2}} = 375\ \mathrm{ppm} = 3.75 \times 10^{-4}\ \mathrm{atm}$ ("ppm" stands for parts per million),
which was the partial pressure of $\mathrm{CO_2}$ at Mauna Loa (Hawaii) in 2003.
Estimate the corresponding pH of rainwater.
Solve this problem using the multivariable version of Newton's method and two variants of Broyden's method.

Solution:
简化记号: (单位均为 $\mathrm{mol/L}$)

$$\begin{aligned}x_1 &:= [\mathrm{H^+}]\\ x_2 &:= [\mathrm{OH^-}]\\ x_3 &:= [\mathrm{HCO_3^-}]\\ x_4 &:= [\mathrm{CO_3^{2-}}]\end{aligned}$$

则化学平衡方程组为:

$$\begin{aligned}10^{-14} &= x_1x_2\\ 10^{-6.3} &= \frac{x_1x_3}{10^{-1.46}\times 3.75\times 10^{-4}}\\ 10^{-10.3} &= \frac{x_1x_4}{x_3}\\ x_1 &= x_2 + x_3 + 2x_4\end{aligned}$$

值得注意的是，若 (x_1, x_2, x_3, x_4) 是方程的解，
则 $(-x_1, -x_2, -x_3, x_4)$ 也近似是方程的解 (因为 x_4 的量级远小于 x_1, x_2, x_3)
因此当我们遇到了 x_1, x_2, x_3 为负数的情况时，可以直接对其取绝对值得到正确结果.

定义 $\mathbb{R}^4$ 上的算子 $f(\cdot)$ 为:

$$f(x) := \begin{bmatrix} x_1x_2 - c_1\\ x_1x_3 - c_2\\ x_1x_4 - c_3x_3\\ x_1 - x_2 - x_3 - 2x_4\end{bmatrix}$$

$$\text{where } x = [x_1, x_2, x_3, x_4]^{\mathrm{T}} \in \mathbb{R}^4_{++} \text{ and } \begin{cases} c_1 = 10^{-14}\\ c_2 = 3.75\times 10^{-11.76}\\ c_3 = 10^{-10.3}\end{cases}$$

Jacobi 矩阵 $Df(x)$ 为:

$$Df(x) := \left[\frac{\partial f_i}{\partial x_j}\right] = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ x_3 & 0 & x_1 & 0 \\ x_4 & 0 & -c_3 & x_1 \\ 1 & -1 & -1 & -2 \end{bmatrix}$$

Matlab 代码:

```
% Constants
c1 = 10^(-14);
c2 = 3.75 * 10^(-11.76);
c3 = 10^(-10.3);

% Define the function f(x)
f = @(x) [
    x(1) * x(2) - c1;                    % Equation 1: x1 * x2 - c1
    x(1) * x(3) - c2;                    % Equation 2: x1 * x3 - c2
    x(1) * x(4) - c3 * x(3);             % Equation 3: x1 * x4 - c3 * x3
    x(1) - x(2) - x(3) - 2 * x(4)        % Equation 4: x1 - x2 - x3 - 2 * x4
];

% Define the Jacobian matrix Df(x)
Df = @(x) [
    x(2), x(1), 0, 0;                % Partial derivatives of Equation 1
    x(3), 0, x(1), 0;                % Partial derivatives of Equation 2
    x(4), 0, -c3, x(1);              % Partial derivatives of Equation 3
    1, -1, -1, -2                    % Partial derivatives of Equation 4
];
```

(a) Newton's Method

Newton 法的迭代格式为:

$$\text{Solve } Df(x^{(k)}) \cdot z = f(x^{(k)}) \text{ to obtain } z^{(k)} = [Df(x^{(k)})]^{-1} f(x^{(k)})$$
$$x^{(k+1)} = x^{(k)} - z^{(k)}$$

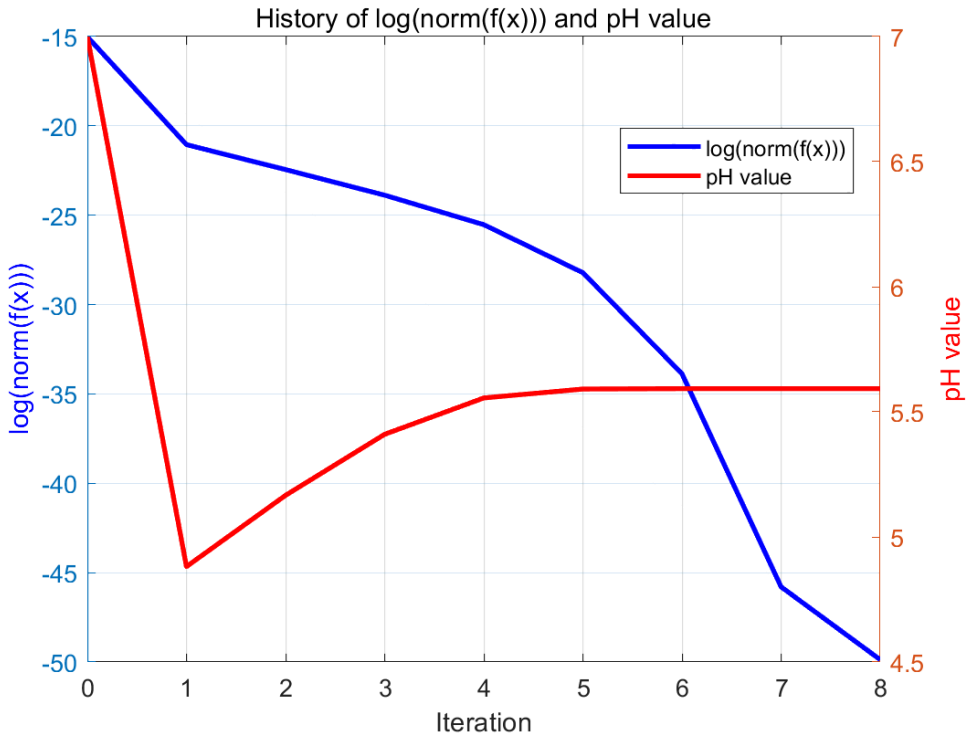
Matlab 代码:

```
% Initial guess for [H+], [OH-], [HCO3-], [CO32-]
x = [1e-7, 1e-7, 1e-7, 1e-7]';

% Maximum number of iterations and convergence tolerance
maxIter = 100;
tol = 1e-12;

% Prepare arrays to store the history of log(norm(f(x))) and pH values
logNormHistory = zeros(maxIter+1, 1);
logNormHistory(1) = log(norm(f(x)));
pHHistory = zeros(maxIter+1, 1);
pHHistory(1) = -log10(x(1));

% Newton's method iteration
for k = 1:maxIter
    % Solve Df(x) * z = f(x) for z
    z = Df(x) \ f(x); % Back-substitute to solve for the step (z)
```

```
    % Update the solution
    x = x - z;

    % Compute the log of the norm of f(x) and pH
    logNormHistory(k+1) = log(norm(f(x)));
    pHHistory(k+1) = -log10(abs(x(1))); % pH = -log10[H+]

    % Check for convergence
    if norm(z) < tol
        fprintf('Converged after %d iterations.\n', k);
        logNormHistory = logNormHistory(1:k+1); % Trim the history arrays
        pHHistory = pHHistory(1:k+1);
        x = abs(x);
        break;
    end
end

% Output the final solution
fprintf('Final concentration values:\n');
fprintf('H+ concentration: %.3e mol/L\n', x(1));
fprintf('OH- concentration: %.3e mol/L\n', x(2));
fprintf('HCO3- concentration: %.3e mol/L\n', x(3));
fprintf('CO32- concentration: %.3e mol/L\n', x(4));

% Calculate pH
pH = -log10(x(1)); % pH = -log10[H+]
fprintf('pH of rainwater: %.3f\n', pH);

% Plot the history of log(norm(f(x))) and pH with different scales
figure;

% Create a plot with two y-axes
yyaxis left
plot(0:length(logNormHistory)-1, logNormHistory, 'b-', 'LineWidth', 2);
ylabel('log(norm(f(x)))', 'Color', 'b');
xlabel('Iteration');
title('History of log(norm(f(x))) and pH value');

yyaxis right
plot(0:length(pHHistory)-1, pHHistory, 'r-', 'LineWidth', 2);
ylabel('pH value', 'Color', 'r');

% Enhance plot appearance
grid on;
legend('log(norm(f(x)))', 'pH value', 'Location', 'best');
```

运行结果:

```
Converged after 8 iterations.
Final concentration values:
H+ concentration: 2.555e-06 mol/L
OH- concentration: 3.914e-09 mol/L
HCO3- concentration: 2.551e-06 mol/L
CO32- concentration: 5.004e-11 mol/L
pH of rainwater: 5.593
```



(b) Good Broyden's Method

好的 Broyden 法的迭代格式为:
(**存疑**: Solve $J_k \cdot z = f(x^{(k)})$ 需要替换为使用 Sherman–Morrison 公式的实现)

$$
\begin{gathered}
J_0 = Df(x^{(0)}) \\
\text{Solve } J_0 \cdot z = f(x^{(0)}) \text{ to obtain } z^{(0)} = J_0^{-1} f(x^{(0)}) \\
x^{(1)} = x^{(0)} - z^{(0)} \\
\hline
\Delta x_k = x^{(k)} - x^{(k-1)} = -z^{(k-1)} \\
\Delta f_k = f(x^{(k)}) - f(x^{(k-1)}) \\
J_k = J_{k-1} + \frac{(\Delta f_k - J_{k-1}\Delta x_k)\Delta x_k^{\mathrm{T}}}{\|\Delta x_k\|^2} \\
\text{Solve } J_k \cdot z = f(x^{(k)}) \text{ to obtain } z^{(k)} = J_k^{-1} f(x^{(k)}) \\
x^{(k+1)} = x^{(k)} - z^{(k)}
\end{gathered}
$$

为保证 J_k 对 $Df(x^{(k)})$ 的近似效果，我们每隔 10 步就计算一次 $J_k = Df(x^{(k)})$.
Matlab 代码如下:

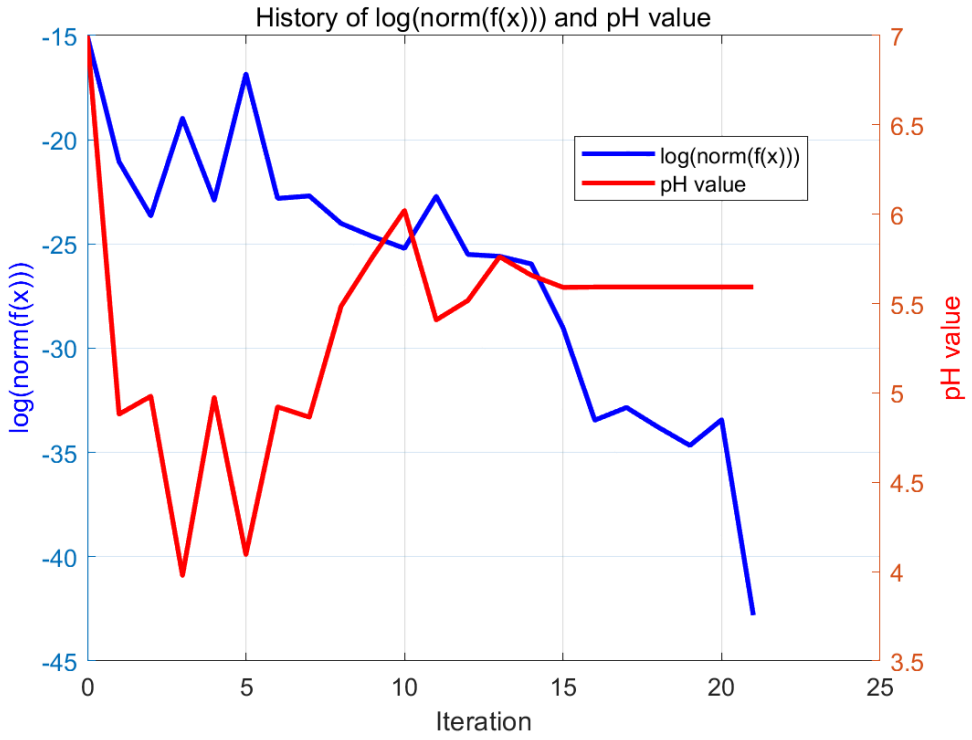
```
% Initial guess for [H+], [OH-], [HCO3-], [CO32-]
x = [1e-7, 1e-7, 1e-7, 1e-7]';
J = Df(x);

% Maximum number of iterations and convergence tolerance
maxIter = 100;
tol = 1e-12;

% Prepare arrays to store the history of log(norm(f(x))) and pH values
logNormHistory = zeros(maxIter+1, 1);
logNormHistory(1) = log(norm(f(x)));
pHHistory = zeros(maxIter+1, 1);
pHHistory(1) = -log10(x(1));
```

```
% First iteration using the Jacobian approximation J_0
fx = f(x);
z = J \ fx; % Back-substitute to solve for the step (z)
x_next = x - z; % Update the solution
fx_next = f(x_next); % Evaluate the function at the current estimate

% Good Broyden's method iteration
for k = 1:maxIter
    % Compute the difference
    delta_f = fx_next - fx;
    delta_x = -z; % delta_x = x_prev - x = -z

    % Calculate the Jacobian
    if mod(k,10) == 0           % Every 10 iterations, recompute the Jacobian exactly
        J = Df(x_next);               % Recompute the Jacobian from scratch
    else
        % Update the Jacobian approximation using the Good Broyden formula
        J = J + (delta_f - J * delta_x) * delta_x' / (delta_x' * delta_x);
    end

    % Solve the system using the updated Jacobian
    z = J \ fx_next; % Back-substitute to solve for the step (z)

    % Update the solution
    x = x_next;                 % Move to the next solution
    x_next = x_next - z;    % Update x_next by subtracting the step (z)
    fx = fx_next;               % Update the function value fx
    fx_next = f(x_next);        % Evaluate the function at the new estimate x_next

    % Compute the log of the norm of f(x) and pH
    logNormHistory(k+1) = log(norm(f(x)));
    pHHistory(k+1) = -log10(abs(x(1))); % pH = -log10[H+]

    % Check for convergence
    if norm(z) < tol
        fprintf('Converged after %d iterations.\n', k);
        logNormHistory = logNormHistory(1:k+1); % Trim the history arrays
        pHHistory = pHHistory(1:k+1);
        x_next = abs(x_next);
        break;
    end
end

% Output the final solution
fprintf('Final concentration values:\n');
fprintf('H+ concentration: %.3e mol/L\n', x_next(1));
fprintf('OH- concentration: %.3e mol/L\n', x_next(2));
fprintf('HCO3- concentration: %.3e mol/L\n', x_next(3));
fprintf('CO32- concentration: %.3e mol/L\n', x_next(4));

% Calculate pH
pH = -log10(x_next(1)); % pH = -log10[H+]
fprintf('pH of rainwater: %.3f\n', pH);

% Plot the history of log(norm(f(x))) and pH with different scales
figure;

% Create a plot with two y-axes
```

```
yyaxis left
plot(0:length(logNormHistory)-1, logNormHistory, 'b-', 'LineWidth', 2);
ylabel('log(norm(f(x)))', 'Color', 'b');
xlabel('Iteration');
title('History of log(norm(f(x))) and pH value');

yyaxis right
plot(0:length(pHHistory)-1, pHHistory, 'r-', 'LineWidth', 2);
ylabel('pH value', 'Color', 'r');

% Enhance plot appearance
grid on;
legend('log(norm(f(x)))', 'pH value', 'Location', 'best');
```

运行结果:

```
Converged after 21 iterations.
Final concentration values:
H+ concentration: 2.555e-06 mol/L
OH- concentration: 3.914e-09 mol/L
HCO3- concentration: 2.551e-06 mol/L
CO32- concentration: 5.004e-11 mol/L
pH of rainwater: 5.593
```



(c) Bad Broyden's Method

为减小 Jacobi 矩阵 $Df(x)$ 的条件数，我们为第四个方程乘上 1×10^{-7} 的系数.
于是 $f(\cdot)$ 的定义变为:

$$f(x) := \begin{bmatrix} x_1x_2 - c_1 \\ x_1x_3 - c_2 \\ x_1x_4 - c_3x_3 \\ (x_1 - x_2 - x_3 - 2x_4) \times 10^{-7} \end{bmatrix}$$

$$\text{where } x = [x_1, x_2, x_3, x_4]^{\mathrm{T}} \in \mathbb{R}^4_{++} \text{ and } \begin{cases} c_1 = 10^{-14} \\ c_2 = 3.75 \times 10^{-11.76} \\ c_3 = 10^{-10.3} \end{cases}$$

对应地，Jacobi 矩阵 $Df(x)$ 的形式为:

$$Df(x) := \left[\frac{\partial f_i}{\partial x_j}\right] = \begin{bmatrix} x_2 & x_1 & 0 & 0 \\ x_3 & 0 & x_1 & 0 \\ x_4 & 0 & -c_3 & x_1 \\ 1 \times 10^{-7} & -1 \times 10^{-7} & -1 \times 10^{-7} & -2 \times 10^{-7} \end{bmatrix}$$

Matlab 代码为:

```
% Constants
c1 = 10^(-14);
c2 = 3.75 * 10^(-11.76);
c3 = 10^(-10.3);

% Define the function f(x)
f = @(x) [
    x(1) * x(2) - c1;                     % Equation 1: x1 * x2 - c1
    x(1) * x(3) - c2;                     % Equation 2: x1 * x3 - c2
    x(1) * x(4) - c3 * x(3);              % Equation 3: x1 * x4 - c3 * x3
    (x(1) - x(2) - x(3) - 2 * x(4)) * 1e-7 % Equation 4: x1 - x2 - x3 - 2 * x4
];

% Define the Jacobian matrix Df(x)
Df = @(x) [
    x(2), x(1), 0, 0;                   % Partial derivatives of Equation 1
    x(3), 0, x(1), 0;                   % Partial derivatives of Equation 2
    x(4), 0, -c3, x(1);                 % Partial derivatives of Equation 3
    1e-7, -1e-7, -1e-7, -2e-7           % Partial derivatives of Equation 4
];
```

坏的 Broyden 法的迭代格式为:

$$\begin{gathered} J_0 = Df(x^{(0)}) \\ \text{Calculate } J_0^{-1} \\ z^{(0)} = J_0^{-1}x^{(0)} \\ x^{(1)} = x^{(0)} - z^{(0)} \\ \hline \Delta x_k = x^{(k)} - x^{(k-1)} = -z^{(k-1)} \\ \Delta f_k = f(x^{(k)}) - f(x^{(k-1)}) \\ J_k^{-1} = J_{k-1}^{-1} + \frac{(\Delta x_k - J_{k-1}^{-1}\Delta f_k)\Delta f_k^{\mathrm{T}}}{\|\Delta f_k\|^2} \\ z^{(k)} = J_k^{-1} f(x^{(k)}) \\ x^{(k+1)} = x^{(k)} - z^{(k)} \end{gathered}$$

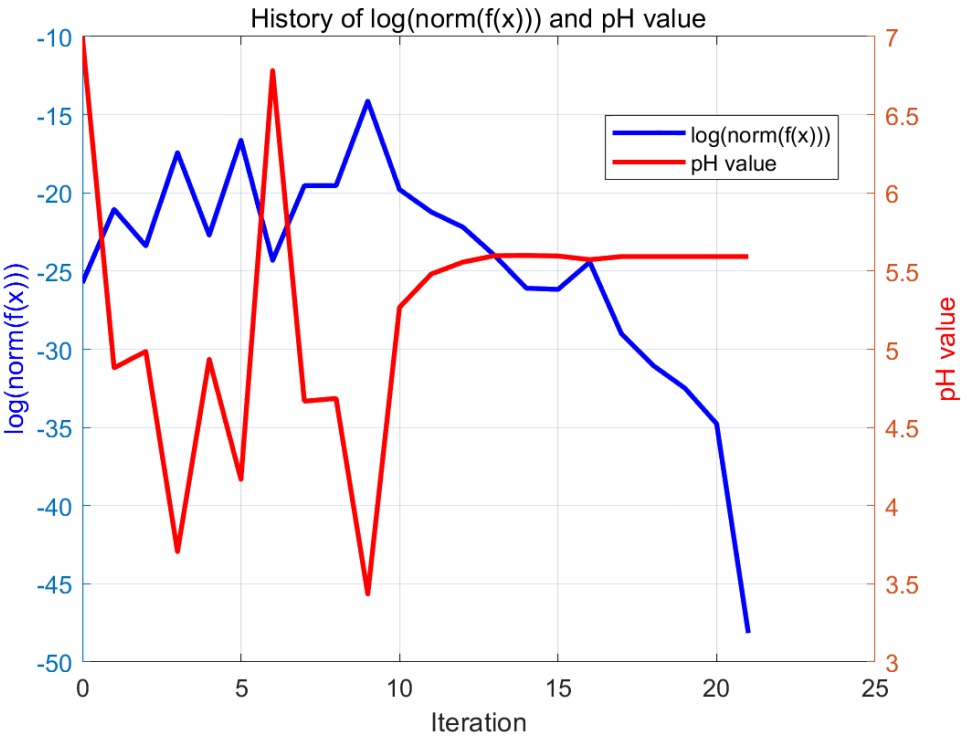
为保证 J_k^{-1} 对 $(Df(x^{(k)}))^{-1}$ 的近似效果，我们每隔 10 步就计算一次 $J_k^{-1} = (Df(x^{(k)}))^{-1}$.
Matlab 代码如下:

```
% Initial guess for [H+], [OH-], [HCO3-], [CO32-]
x = [1e-7, 1e-7, 1e-7, 1e-7]';
J = Df(x);
J_inv = inv(J);

% Maximum number of iterations and convergence tolerance
maxIter = 100;
tol = 1e-12;

% Prepare arrays to store the history of log(norm(f(x))) and pH values
logNormHistory = zeros(maxIter+1, 1);
logNormHistory(1) = log(norm(f(x)));
pHHistory = zeros(maxIter+1, 1);
pHHistory(1) = -log10(x(1));

% First iteration using the Jacobian approximation J_0
fx = f(x);
z = J \ fx; % Back-substitute to solve for the step (z)
x_next = x - z; % Update the solution
fx_next = f(x_next); % Evaluate the function at the current estimate

% Good Broyden's method iteration
for k = 1:maxIter
    % Compute the difference
    delta_f = fx_next - fx;
    delta_x = -z; % delta_x = x_prev - x = -z

    % Update the Jacobian using the Bad Broyden formula
    if mod(k,10) == 0           % Every 10 iterations, recompute the Jacobian exactly
        J = Df(x_next);
        J_inv = inv(J);               % Recompute the Jacobian from scratch
        % Solve the system using the updated Jacobian
        z = J \ fx_next; % Back-substitute to solve for the step (z)
    else
        % Update the Jacobian approximation using the formula
        J_inv = J_inv + (delta_x - J_inv * delta_f) * delta_f' / (delta_f' * delta_f);
        z = J_inv * fx_next;
    end

    % Update the solution
    x = x_next;                 % Move to the next solution
    x_next = x_next - z;    % Update x_next by subtracting the step (z)
    fx = fx_next;               % Update the function value fx
    fx_next = f(x_next);        % Evaluate the function at the new estimate x_next

    % Compute the log of the norm of f(x) and pH
    logNormHistory(k+1) = log(norm(f(x)));
    pHHistory(k+1) = -log10(abs(x(1))); % pH = -log10[H+]

    % Check for convergence
    if norm(z) < tol
        fprintf('Converged after %d iterations.\n', k);
        logNormHistory = logNormHistory(1:k+1); % Trim the history arrays
        pHHistory = pHHistory(1:k+1);
        x_next = abs(x_next);
        break;
    end
end
```

```
% Output the final solution
fprintf('Final concentration values:\n');
fprintf('H+ concentration: %.3e mol/L\n', x_next(1));
fprintf('OH- concentration: %.3e mol/L\n', x_next(2));
fprintf('HCO3- concentration: %.3e mol/L\n', x_next(3));
fprintf('CO32- concentration: %.3e mol/L\n', x_next(4));

% Calculate pH
pH = -log10(x_next(1)); % pH = -log10[H+]
fprintf('pH of rainwater: %.3f\n', pH);

% Plot the history of log(norm(f(x))) and pH with different scales
figure;

% Create a plot with two y-axes
yyaxis left
plot(0:length(logNormHistory)-1, logNormHistory, 'b-', 'LineWidth', 2);
ylabel('log(norm(f(x)))', 'Color', 'b');
xlabel('Iteration');
title('History of log(norm(f(x))) and pH value');

yyaxis right
plot(0:length(pHHistory)-1, pHHistory, 'r-', 'LineWidth', 2);
ylabel('pH value', 'Color', 'r');

% Enhance plot appearance
grid on;
legend('log(norm(f(x)))', 'pH value', 'Location', 'best');
```

运行结果:

```
Converged after 21 iterations.
Final concentration values:
H+ concentration: 2.555e-06 mol/L
OH- concentration: 3.914e-09 mol/L
HCO3- concentration: 2.551e-06 mol/L
CO32- concentration: 5.004e-11 mol/L
pH of rainwater: 5.593
```



Problem 5 (optional)

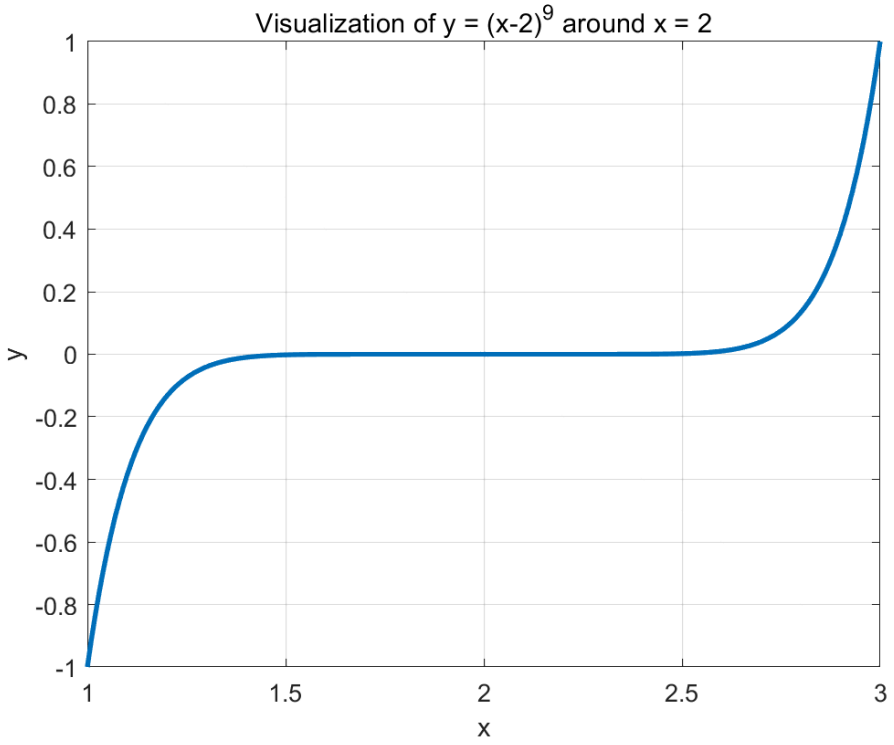
Visualize the curve $y = (x-2)^9$ around $x = 2$,
where the function $f(x) = (x-2)^9$ is evaluated through an expanded form.
What is the attainable accuracy if bisection is used to find the root of this function?

- **二分法的缺点:**
  设 l, r 分别为区间左右端点.
  当 $f(l)$ 和 $f(r)$ 至少有一个比较小时，相对误差会比较大，导致符号判断不准确.

Solution:
通过二项展开式绘制 $y = (x-2)^9$ 在 $[1, 3]$ 区间上的图像的 Matlab 代码如下:

```
% Define the range of x values around x=2
x = linspace(1, 3, 200);

% Compute the expanded form of (x - 2)^9 using the binomial expansion
n = 9;  % Degree of the polynomial
expanded_form = zeros(size(x));

for k = 0:n
    % Evaluate the expanded form
    expanded_form = expanded_form + nchoosek(n, k) * (-2)^(n-k) * x.^k;
end

% Plot the result
figure;
plot(x, expanded_form, 'LineWidth', 2);
title('Visualization of y = (x-2)^9 around x = 2');
xlabel('x');
ylabel('y');
grid on;
hold off;
```

运行结果:



假设二分法的初始区间是 (a, b)
其中 $a < 2 < b$，这保证了 $f(a)f(b) < 0$

- 最好的情况: $\frac{a+b}{2} = 2$，此时二分法得到的解 $x_{\text{approx}} = 2$ 是完全准确的.
- 最差的情况: $(x_{\text{approx}} - 2)^9 = \text{eps} = 2^{-52}$ (双精度浮点数的机器精度)
  在忽略舍入误差的情况下，解得 $|x_{\text{approx}} - 2| = 2^{-52/9} \approx 0.0182$
  表明二分法得到的近似解 x_{approx} 与精确解 $x = 2$ 的差距至多约为 0.0182

Problem 6 (optional)

Give a local convergence analysis on Steffensen's method.

- 更严格的解法参见 **Final Exam Problem 2**

Solution: (reference)
Steffensen 方法是 Newton 法的简化版本，它通过以下公式近似导数:

$$f'(x_k) \approx \frac{f(x_k + f(x_k)) - f(x_k)}{f(x_k)}$$

于是 Newton 法的迭代格式 $x_{k+1} = x_k - \frac{f(x_k)}{f'(x_k)}$ 变为:

$$x_{k+1} = x_k - \frac{(f(x_k))^2}{f(x_k + f(x_k)) - f(x_k)}$$

注意到:

$$\begin{aligned} f(x + f(x)) - f(x) &= f'(x)(x + f(x) - x) + \frac{1}{2}f''(\xi)(x + f(x) - x)^2 \\ &= f'(x)f(x) + \frac{1}{2}f''(\xi)(f(x))^2 \quad (\text{for some } \xi \in (x, x + f(x))) \end{aligned}$$

简单起见，假设 $f(x_\star) = 0$ 且 $f'(x_\star) \neq 0$ (即 $x_\star$ 为单重根)，则我们有:

$$
\begin{aligned}
f(x_k) &= f(x_\star) + f'(x_\star)(x_k - x_\star) + \frac{1}{2}f''(x_\star)(x_k - x_\star)^2 + O((x_k - x_\star)^3) \\
&= f'(x_\star)(x_k - x_\star) + \frac{1}{2}f''(x_\star)(x_k - x_\star)^2 + O((x_k - x_\star)^3)
\end{aligned}
$$

于是我们有:

$$
\begin{aligned}
x_{k+1} - x_\star &= x_k - \frac{(f(x_k))^2}{f(x_k + f(x_k)) - f(x_k)} - x_\star \\
&= (x_k - x_\star) - \frac{(f(x_k))^2}{f'(x_k)f(x_k) + \frac{1}{2}f''(\xi)(f(x_k))^2} \\
&= (x_k - x_\star) - \frac{f(x_k)}{f'(x_k) + \frac{1}{2}f''(\xi)f(x_k)} \\
&= (x_k - x_\star) - \frac{f'(x_\star)(x_k - x_\star) + \frac{1}{2}f''(x_\star)(x_k - x_\star)^2 + O((x_k - x_\star)^3)}{f'(x_\star) + f''(x_\star)(x_k - x_\star) + \frac{1}{2}f''(\xi)f'(x_\star)(x_k - x_\star) + O((x_k - x_\star)^2)} \\
&= \frac{\frac{1}{2}f''(x_\star)(x_k - x_\star)^2 + \frac{1}{2}f''(\xi)f'(x_\star)(x_k - x_\star)^2 + O((x_k - x_\star)^3)}{f'(x_\star) + f''(x_\star)(x_k - x_\star) + \frac{1}{2}f''(\xi)f'(x_\star)(x_k - x_\star) + O((x_k - x_\star)^2)} \\
&= \frac{f''(x_\star) + f''(\xi)f'(x_\star) + O(x_k - x_\star)}{2f'(x_\star) + 2f''(x_\star)(x_k - x_\star) + f''(\xi)f'(x_\star)(x_k - x_\star) + O((x_k - x_\star)^2)}(x_k - x_\star)^2 \quad (\text{note that } f'(x_k) \neq 0) \\
&\to \frac{f''(x_\star) + f''(\xi)f'(x_\star)}{2f'(x_\star)}(x_k - x_\star)^2 \quad (\text{if } x_k \to x_\star)
\end{aligned}
$$

这表明 Steffensen 方法具有局部二次收敛性.

Problem 7 (optional)

The Lambert W-function, $y = W(x)$, is the inverse function of $x = y\exp(y)$ for $y \in [-1, \infty)$
Make a plot of the Lambert W-function, with at least 100 equally spaced sampling points over x.
Verify your plot using the graph of $x = y\exp(y)$

Solution: (reference)
注意到 $\frac{\mathrm{d}x}{\mathrm{d}y} = (y+1)e^y \geq 0$ (当且仅当 $y = -1$ 时取等)
于是 $x = y\exp(y)$ 是关于 $y \in [-1, \infty)$ 严格单调递增的，因而在 $[-1, \infty)$ 上存在反函数 $y = W(x)$.
显然 $\mathrm{dom}(W) = [-e^{-1}, \infty)$

对于任意 $\alpha \in \mathrm{dom}(W) = [-e^{-1}, \infty)$
我们可以定义 $f(y) = y\exp(y) - \alpha$ 并使用 Newton 法求解它的根:

$$
\begin{aligned}
y_{k+1} &= y_k - \frac{f(y_k)}{f'(y_k)} \\
&= y_k - \frac{y_k\exp(y_k) - \alpha}{(y_k+1)\exp(y_k)} \\
&= y_k - \frac{y_k}{y_k+1} + \frac{\alpha}{(y_k+1)\exp(y_k)} \\
&= \frac{y_k^2 + \alpha\exp(-y_k)}{y_k+1}
\end{aligned}
$$

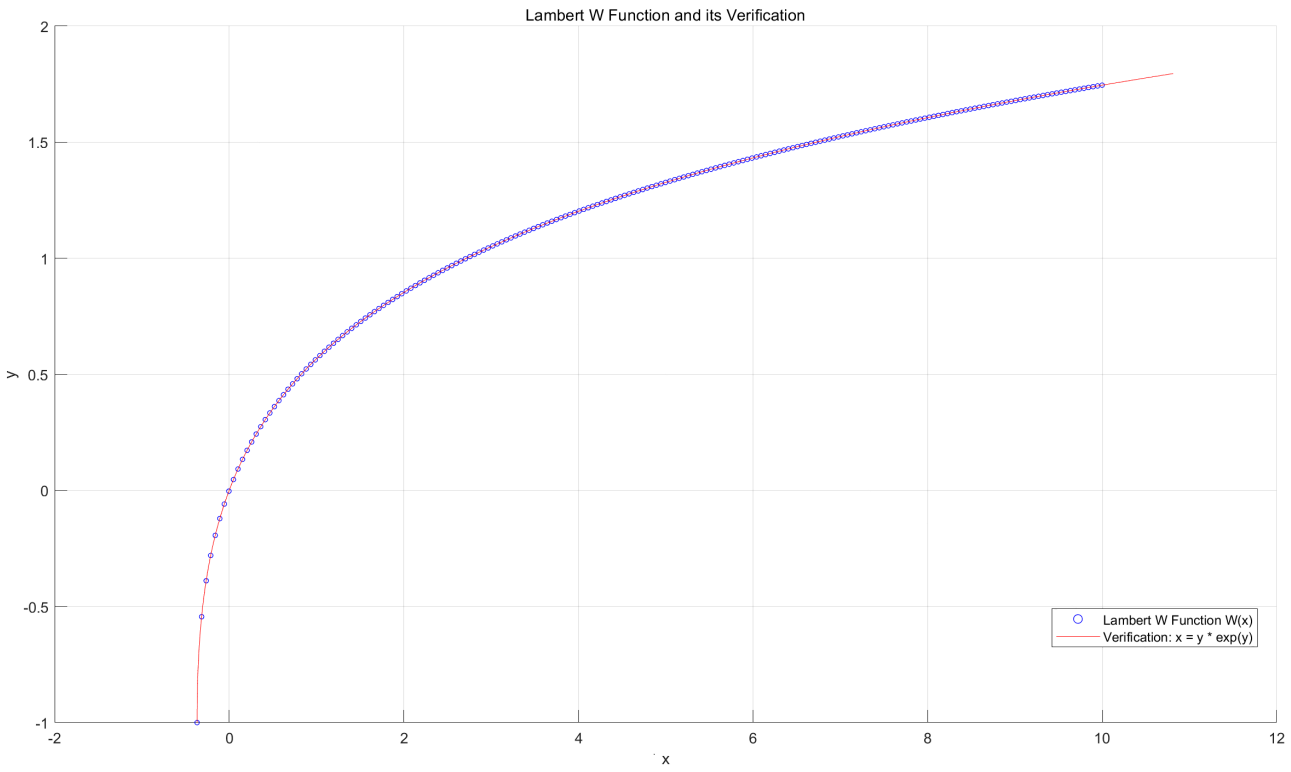
假设在 $[-e^{-1}, \infty)$ 上等距取样了 n 个 x 值: $x^{(1)}, \ldots, x^{(n)}$
我们可以顺序计算 $y^{(1)}, \ldots, y^{(n)}$，每次将 $y^{(k-1)}$ 作为计算 $y^{(k)}$ 的 Newton 迭代的初值.
此外，$W(-e^{-1}) = -1$ 和 $W(0) = 0$ 也可辅助我们设置初值.
Matlab 代码如下:

```
% Sampling range
x_min = -exp(-1);
```

```
x_max = 10;
n_points = 200; % Number of sampling points
tol = 1e-10;
max_iter = 100;

% Generate equally spaced x values
x_vals = linspace(x_min, x_max, n_points);
y_vals = zeros(1, n_points);
y_vals(1) = -1; % Note that W(-exp(-1)) = -1

% Newton's method to solve f(y) = y*exp(y) - x = 0
for i = 2:n_points
    x = x_vals(i);

    if x < 0 || abs(y_vals(i-1) + 1) < 1e-10
        % Note that W(0) = 0
        y = 0;
    else
        % Use the convergence value of the previous Newton process as initial guess
        y = y_vals(i-1);
    end

    % Newton's method iteration
    for iter = 1:max_iter
        % Update y using Newton's method
        y_new = (y^2 + x*exp(-y)) / (y+1);

        % Check for convergence
        if abs(y_new - y) < tol
            break;
        end

        y = y_new;
    end

    if iter == max_iter
        fprintf("Warning: %d-th point's maximum number of iteration is reached!\n", i);
    end

    % Store the result
    y_vals(i) = y_new;
end

% Plot the Lambert W function (y vs x)
figure;
scatter(x_vals, y_vals, 10, 'b');
hold on;

% Verify the plot using x = y * exp(y)
n_verify = 1000;
y_vals_verify = linspace(-1, y_vals(end)+5e-2, n_points);
x_vals_verify = y_vals_verify .* exp(y_vals_verify);
plot(x_vals_verify, y_vals_verify, 'r')

% Add labels and legend
xlabel('x');
ylabel('y');
legend('Lambert W Function W(x)', 'Verification: x = y * exp(y)', 'Location', 'Best');
```

```
title('Lambert W Function and its Verification');
grid on;
```

运行结果:



The End